

UNIVERSITÉ LIBRE DE BRUXELLES

Faculté des Sciences

Département d'Informatique

An Experimental Framework for Evaluating Traffic Deviation Plans

CALLENS HUGO

Promotors :

PROF. GIANLUCA BONTEMPI

ELADIO MONTERO PORRAS

DAVIDE ANDREA GUASTELLA

Master Thesis in Computer Sciences

Academic year 2025 – 2026

Contents

1 Introduction

1.1 Urban context and motivation

Introduce the problem of traffic disruptions (planned works, incidents, etc.) and explain why selecting detours is difficult due to network topology and traffic conditions. It motivates the need for a decision-support approach that can compare multiple recovery plans in a realistic way rather than relying on intuitions and simple metrics.

1.1.1 Digital Twin

A digital twin is a data-driven virtual representation of real-world entities designed to mirror a physical object or system throughout its lifecycle. The virtual part is updated with real-time data gotten from sensors or other sources making it possible to link the physical and digital parts. [defDT, IBMDT]

A digital twin consists of three components [DT_health]:

1. **Physical entity:** The real-world object or system being modeled (e.g., traffic, a plant)
2. **Virtual representation:** A model that replicates the behavior of the physical entity.
3. **Connection:** The link between the physical system and the virtual system making it possible to update the physical system by exchanging information via this real-time connection.

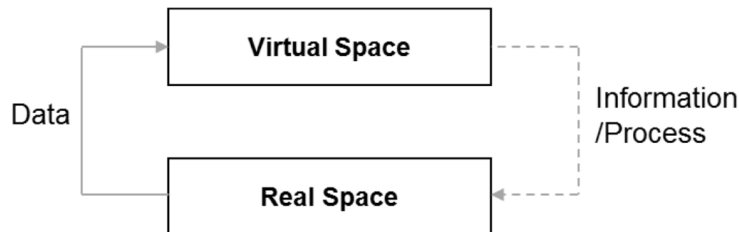


Figure 1: Illustration of a digital twin from [digital_twin]

Digital twins receive continuously streams of data from sensors or other sources ensuring that the virtual model replicates the current state of the physical object. The main goals for a digital twin are to simulate the physical entity and predict what could happen in order to optimize or predict failures. Urban traffic networks, which are both data-rich and fast-evolving, provide fertile ground for the deployment of digital twins (see ??). Digital twins in traffic can potentially simulate and optimize traffic flows, supporting smarter city planning and real-time operations. A detailed discussion of modern digital twin solutions in traffic management is provided in ??.

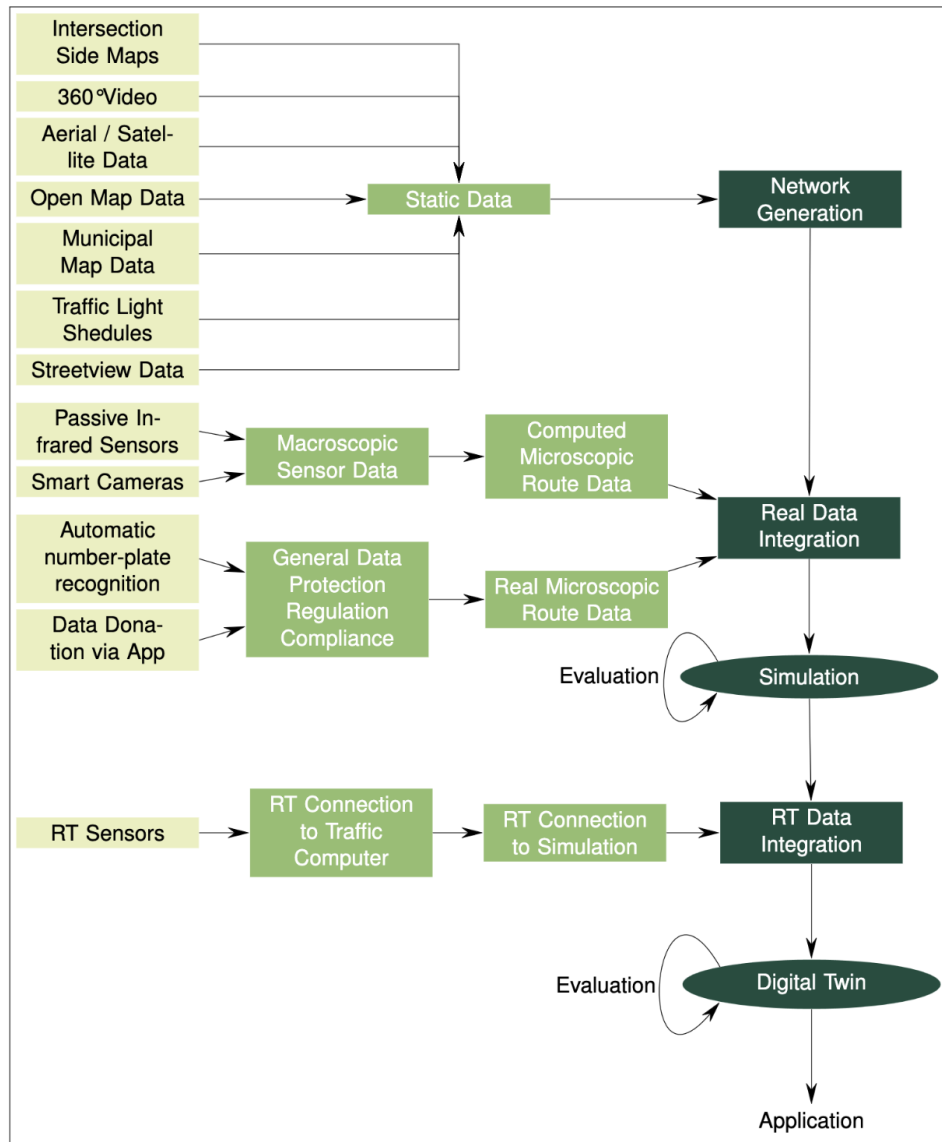


Figure 2: xample workflow for driving SUMO towards a digital twin. Various static and real-time data sources (maps, sensors, cameras) are integrated into simulation models to continuously update and calibrate the digital twin for real-world application. Source : [sumo_twin]

1.1.2 Context

In this project, a digital twin acts as a precise and efficient method to simulate traffic based on real-world data. Digital twins leverage continuous data streams (for example, from vehicle counting devices) to create an up-to-date virtual model of physical traffic flows. Accurate traffic simulation relies on high-resolution data, typically gathered by a variety of counting devices such as inductive loop traffic detectors, Bluetooth recognition systems and ANPR (Automatic Number Plate Recognition) cameras [**counting_devices**]. These devices measure the number of vehicles passing specific locations over time, producing the data needed for dynamic modeling and simulation.

Through the aggregation and analysis of such data, it becomes possible to estimate traffic density and infer vehicle movement patterns across a road network. In this framework, physical traffic corresponds to the physical entity of the digital twin, real-time vehicle counts represent the virtual layer and the counting devices themselves serve as the communication link between the physical and digital components.

Traffic simulation is particularly vital in urban environments such as Brussels, where the frequent congestion and ongoing roadworks demand dynamic management of the transport network. Planned deviations due to construction may alter typical traffic flows and produce unexpected bottlenecks. Accurate simulation enables city planners to evaluate alternative deviation plans and identify those that minimize congestion and disruption. Additionally, the direct and indirect effects of traffic interventions, such as road closures, can be promptly assessed.

1.2 Problem statement

The thesis problem : Given a baseline traffic situation and a disruptive event (road closure) the goal is to generate feasible deviation plans and assess their impact on traffic in order to recommend the best plan to the urban planner.

1.2.1 Objective

The main objective of this work is the development of a web-based application that reproduces and extends the functionalities of an existing tool, TrafficTwin¹ [**SIM_param**]. This modernization enhances accessibility and usability. The work is divided into several sub-goals:

TO BE WRITTEN

Ultimately, the goal is to provide city planners and traffic managers with actionable, data-driven tools for congestion mitigation and urban mobility planning.

1.3 Research questions

Present the research questions that will be addressed in the thesis. The main one is “How deviation plans should be evaluated?” and the subquestions are “What are the key factors influencing the evaluation of deviation plans?” and “How can we develop a

¹<https://traffictwin.ulb.be>

decision-support approach to compare multiple deviation plans?”.

1.4 Contributions and thesis outline

Summarize the contributions (end-to-end deviation plan assessment pipeline, candidate plan generation strategies, Monte Carlo evaluation protocol). It ends with a short outline of the thesis chapters.

2 Foundations and Related Work

2.1 Traffic digital twins for decision support

2.1.1 Digital twins in traffic simulation

A traffic digital twin is useful here less as a real-time control system than as a planning environment in which a road manager can ask a concrete counterfactual question: what happens if a road segment is closed and traffic is assigned to one of several deviation plans? The literature on digital twins for mobility emphasizes this ability to reproduce a road network, ingest transport data, simulate alternative scenarios and support decision making before actions are deployed in the physical network [**data_mining_dt**, **intelligent_transport**]. This thesis uses that same decision-support idea, but it implements it as an offline experimental pipeline rather than as a continuously synchronized operational twin.

The implemented system therefore separates the physical city from the experimental model. The physical layer is represented by a SUMO road network and traffic analysis zones, the scenario layer generates synthetic, network-aware origin-destination demand, the planning layer converts a selected closure into candidate detour paths and the evaluation layer simulates and ranks those candidates. This is narrower than a full urban digital twin with live sensor ingestion, privacy-sensitive trajectory mining and low-latency feedback loops. Those capabilities are important in operational ITS deployments [**data_mining_dt**, **intelligent_transport**], but the contribution of this work is the reproducible comparison of deviation policies under controlled assumptions.

The result is a decision-support digital twin prototype for roadwork planning. Its purpose is not to predict the exact state of Brussels traffic at a particular minute. Instead, it provides a repeatable environment for testing how different detour definitions change route assignment, vehicle time loss, travel time and the number of affected users. This framing is important because roadwork decisions are often made before the disruption occurs. The planner needs evidence about candidate plans, not only a live dashboard after congestion has already formed.

2.1.2 Simulation and mobility

Traffic simulation is the mechanism that turns the digital-twin idea into an experiment. The common theme across simulation-based traffic studies is that a proposed intervention can be tested in a controlled environment before it is implemented on the road. Prior work uses this principle for adaptive signal control, co-simulation and AI-based traffic management [**ai_driven_simulation**, **cosimulation**, **traffic_light_simulation**]. Those studies are useful background, but their intervention is different from the one studied here. This thesis does not learn traffic-light policies or run multiple simulators together. It evaluates fixed road-closure responses.

The simulation problem addressed here has three practical requirements. First, the same demand must be reused across alternatives so that candidate plans are compared

on equal terms. Second, the route assignment must be allowed to vary from run to run, because different drivers can perceive different costs and choose different paths. Third, the final recommendation must combine several outputs, since a plan that reduces one metric may worsen another. The pipeline answers these requirements by generating one reproducible `trips.xml`, repeatedly routing it with `duarouter`, rewriting affected routes according to each candidate plan and evaluating all conditions with paired statistics and multi-criteria ranking.

In this setting, simulation is not used only for visualization. It is the measurement instrument of the thesis. The output of each SUMO run is parsed into quantitative indicators and the evaluator uses the distribution of those indicators over repeated runs to decide which deviation plan is most robust for the selected closure.

2.1.3 Positioning of this work

This thesis is positioned between three existing lines of work. The first is SUMO-based traffic model construction, where tutorials and case studies explain how road networks, demand and simulation files can be assembled for urban experiments [`sumo_tutorial`, `sparse_data_SUMO`]. The second is digital-twin research for transport, which motivates using simulation models to support planning decisions in complex road networks [`intelligent_transport`, `digital_twin_transport`]. The third is *TrafficTwin*, an earlier interactive tool for designing and evaluating deviation plans [`SIM_param`].

The present project does not attempt to build a complete real-time Brussels twin. It contributes a specific experimental framework around deviation plans. The framework includes a synthetic demand generator, automatic candidate-plan generation, Monte Carlo simulation with stochastic routing and a multi-criteria evaluator. This scope matches the practical decision facing an urban planner: given a closed road edge and a set of feasible detours, compare the alternatives in a way that is reproducible, statistically aware and explainable.

2.2 Traffic simulation tools

Traffic congestion is one of the major challenges faced by modern cities and is expected to get worse with increasing population and vehicle usage [`increasing_traffic`] [`increasing_cars`]. Traffic simulation tools have been developed as mathematical and computational models that replicate real-world traffic behavior and dynamics [`TST_def`]. These tools help transportation planners and engineers to optimize road networks designs or estimate traffic-related emissions.

Traffic simulations are commonly categorized into three main types based on the level of detail desired [`TST_types`]:

1. **Microscopic Simulation:** Models each traffic participant (e.g., vehicles, pedestrians) individually. Agents continuously in time and space respond to their immediate environment. The global traffic dynamics is the result of the sum of these individual decisions and behaviors.
2. **Mesoscopic Simulation:** Still represents individual agents, but their behavior is

determined by average traffic conditions like speed or density. Agents only interact directly at key points in the network, such as intersections.

3. **Macroscopic Simulation:** Looks at traffic as a whole rather than focusing on individual vehicles.

Choosing a simulation type entails balancing computational cost and realism. For instance, microscopic simulation is ideal for projects that require modeling of individual behaviors or the testing of interventions at the street or intersection level, which is often necessary for a dense urban environment such as Brussels. In contrast, a macroscopic simulation might suffice for citywide traffic forecasting or strategic planning, where the focus is on broader patterns rather than local details.

2.2.1 SUMO

SUMO (Simulation of Urban MObility) is an open-source, highly portable, microscopic and continuous multi-modal traffic simulation tool. It allows detailed modeling of individual vehicles, each following its own route through a specified road network. As a microscopic simulator, SUMO treats every vehicle as an individual entity with specific behaviors and interactions. The tool is multi-modal enabling the simulation of diverse traffic participants including cars, public transport vehicles and pedestrians [SUMO_eclipse] [SUMO_homepage] [SUMO_doc_glance].

SUMO provides some tools to setup the simulation and managing the data of the road network [SUMO_eclipse] [SUMO_doc_glance]. In this project, the following components of the SUMO ecosystem are particularly relevant:

- **sumo** : Executes command-line-based microscopic traffic simulations without graphical visualization.
- **netconvert** : Imports and converts road network data from various formats into the internal SUMO network format.
- **duarouter** : Computes optimal routes across the network based on demand inputs and performs Dynamic User Assignment (DUA).

SUMO has been used extensively to model real-world traffic systems in diverse urban contexts, including cities such as Berlin, Dublin and Turin [SUMO_scenarios]. Each scenario adapts to the specific traffic dynamics and urban characteristics of its location :

1. **Berlin:** Berlin has served for large-scale traffic simulations using SUMO. The "24 Hours of Traffic in Berlin" scenario captures over 2.2 million trips within an $800km^2$ area. This model enhances traditional traffic simulations by incorporating motorized private vehicles alongside realistic bus schedules derived from actual General Transit Feed Specification (GTFS) data, representing 89% of the city's bus lines. This scenario allows for holistic analyses that combine traffic, communication (including V2X and LTE/5G) and application simulations, making it suited for smart mobility researches. [berlin_scenario]
2. **Dublin:** The study by Gueriau and Dusparic [dublin_scenario] leverages SUMO to simulate and analyze the impact of connected and autonomous vehicles (CAVs) on real

Irish road networks, covering urban, national and motorway settings. Using detailed demand and network data from Dublin and surrounding areas, the research quantifies how varying CAV penetration rates affect both traffic efficiency and open dataset for benchmarking the effects of emerging vehicle technologies in mixed traffic environments.

3. **Turin:** Leveraging real-world data, the TuSTScenario [**turin_scenario**] provides a highly detailed SUMO-based simulation of metropolitan Turin, covering over $602.61km^2$. The model integrates structural road information from OpenStreetMap, validated average speeds from Google APIs, actual traffic light programming and real trip demands from local mobility services. With more than 2.2 million simulated daily trips and validated against city sensor data, this scenario enables realistic studies of urban traffic dynamics and congestion patterns.

Brussels is also a large city warranting similarly large-scale simulations. However, the focus of this study is distinct: rather than prioritizing multi-modal integration or CAV impact, we specifically investigate the planning and management of roadworks within the Brussels network. The goal is to demonstrate how such interventions can be realistically modeled, analyzed, and optimized in SUMO for modern urban infrastructure management.

Beyond traditional traffic flow analysis SUMO has been used to evaluate the effects of technologies like virtual traffic lights communication systems where cars directly communicate with each other. [**SUMO_VTL**] This study showed that replacing traffic lights with virtual ones can reduce both congestion and the likelihood of traffic accidents. Moreover SUMO can be used in environmental research, for instance in assessing the impact of traffic-related emissions on urban air quality. In such cases traffic flows are generated from sensor data to estimate real-world emission patterns [**SUMO_emission**].

One of the strengths of SUMO is its deterministic core which allows reproducibility of experiments. However, this deterministic nature can be a limitation when it comes to capturing the variability in human driving behavior which requires the integration of stochastic elements [**SUMO_sumo**].

2.2.2 Alternatives to SUMO

Several other traffic simulation tools can serve as alternatives to SUMO, each with different features, advantages and limitations :

1. PTV Vissim ² : PTV-Vissim is a widely used microscopic traffic simulation software that also supports multi-modal transport systems. It offers a broad set of features comparable to those of SUMO. However, it is a proprietary tool that requires a commercial license, which can restrict its accessibility and limit the ability to implement custom modifications at the code level.
2. Aimsun³ : Aimsun is a commercial traffic simulation platform that supports microscopic, mesoscopic, and macroscopic modeling. While it is used in various contexts, its commercial nature introduces licensing costs and reduces flexibility for developers who require deeper code-level access and customization.

²<https://www.ptvgroup.com/en/products/ptv-vissim>

³<https://docs.aimsun.com/>

3. CityFlow [**cityflow**] : CityFlow is an open-source microscopic simulator designed for efficient, large-scale urban traffic simulations, capable of handling thousands of intersections with high computational efficiency. Its architecture is highly optimized for speed and parallel execution, making it over twenty times faster than SUMO in large-scale benchmarks [**cityflow_rl**]. CityFlow is particularly popular in the field of artificial intelligence and reinforcement learning, serving as a benchmark for multi-agent adaptive traffic signal control and dynamic routing tasks. However, CityFlow remains limited compared to SUMO in certain areas: it offers fewer out-of-the-box features for conventional traffic simulation, has a less mature ecosystem, and does not natively support the simulation of public transportation modes such as buses, trams, or trains, which restricts its suitability for comprehensive, multi-modal urban simulations.

2.2.3 Why prefer SUMO

SUMO was selected for this project based on the following criteria:

- **Open-source** : SUMO’s open-source status allows users to freely modify, adapt, and extend the software, making it suitable for research and academic use where flexibility and transparency are important. This degree of code-level access is not available in proprietary tools like PTV Vissim or Aimsun.
- **Community and documentation** : SUMO benefits from a large active community that contributes regularly to new features or bug fixes. In addition, it offers extensive documentation and a wide range of tutorials, which significantly lower the learning curve for new users.
- **Microscopic and multi-modal simulation** : SUMO provides detailed agent-based (microscopic) modeling of cars, public transport, and pedestrians. While other simulators such as Vissim and Aimsun also support microscopic and multi-modal simulation, SUMO’s implementation is open, which enables deeper customization and integration for research needs.
- **Integration and flexibility** : SUMO’s design and command-line tools (like **netconvert** and **duarouter**) facilitates working on various data sources making it easy to connect with external components.

Commercial tools like PTV Vissim and Aimsun offer advanced graphical user interfaces and support; however, their proprietary nature limits customizability and long-term usability in research settings. CityFlow, while open-source and scalable, lacks SUMO’s extensive documentation and wider range of traffic simulation features relevant for this project’s objectives.

2.3 Stochastic simulation

In a traffic-management digital twin, uncertainty is part of the phenomenon being modeled rather than a secondary implementation detail. Traffic demand, vehicle insertion, lane changing, speed preferences, and route choices all contain stochastic components, and small changes in these components can propagate into visibly different congestion pat-

terns [**digital_twin_transport**, **flow_twin**, **stochastic_macroscopic**]. The Federal Highway Administration (FHWA) therefore recommends that microsimulation alternatives be evaluated with multiple random seeds because stochastic model variation can affect the statistical validity of observed differences between alternatives [**fhwa_microsim_2019**].

Stochastic simulation treats selected inputs or model mechanisms as random variables and observes the distribution of outcomes over repeated replications [**application_stochastic_risk**]. For this work, the decision output is therefore not a single deterministic delay value for each plan, but a set of paired delay, travel-time and impacted-user observations across simulation seeds. The evaluator uses these observations to estimate means, confidence intervals, and ranking robustness.

2.3.1 Theoretical foundations of the Monte Carlo method

The Monte Carlo method estimates quantities of interest by repeated random sampling. If each simulation run is treated as one realization of an output metric, then repeated runs approximate the distribution of that metric under the model’s stochastic mechanisms.

Law of Large Numbers and convergence. Let $X_1, \dots, X_n$ be independent replications of an output metric X with finite mean μ . The sample mean $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ converges to μ as n increases. In the traffic-simulation setting, X_i is the metric produced by run i with a distinct random seed. This is why additional replications stabilize the estimated mean, provided the model configuration is otherwise held fixed [**law_kelton**, **number_of_runs_traffic**].

Confidence intervals. A sample mean alone does not communicate how much uncertainty remains after n runs. For a metric with sample standard deviation s , the usual small-sample interval for the mean is

$$\bar{X}_n \pm t_{0.975, n-1} \frac{s}{\sqrt{n}},$$

where $t_{0.975, n-1}$ is the Student- t critical value. This is the form used in the evaluator’s 95% confidence intervals and is also the basis of the run-count formulas used in traffic microsimulation guidance [**law_kelton**, **txdot_number_runs**].

Paired design for plan comparison. A deviation plan and its baseline are compared under the same seed whenever possible. This paired design controls for seed-specific variation in demand realization and driver behavior: the estimator is applied to

$$\Delta^{(r)} = X_{\text{plan}}^{(r)} - X_{\text{baseline}}^{(r)}$$

rather than to two unrelated samples. Preserving the run-by-run difference is especially important when the random seed affects vehicle insertion or behavioral choices in both scenarios. The paired series is then summarized with the same confidence-interval machinery used for other stochastic simulation outputs.

2.3.2 Randomness in microscopic traffic simulation

SUMO documents stochasticity as a core mechanism for reproducing reality in a simulation scenario. The seed initializes the random-number generators and a fixed seed makes a run

deterministic and reproducible. Changing the seed changes the random sequence used by stochastic model components.

Several SUMO mechanisms are directly relevant here. **Speed distributions** allow vehicles to draw heterogeneous speed preferences through the `speedFactor` parameter, SUMO states that this random value is selected when each vehicle is created and that a default speed distribution is active. **Car-following stochasticity** enters through model parameters such as `sigma`, which introduces random variation in driver speed behavior. **Departure-time variation** can be added with random offsets or randomized flows, which prevents unrealistically synchronized departures and better represents variable demand realization [`sumo_randomness`].

2.3.3 Determining the number of simulation runs

The required number of replications depends on output variability and the precision required by the decision. The Texas Department of Transportation’s (TxDOT) microsimulation procedures, following FHWA guidance, state that random seeds represent natural variation in driver behavior and vehicle entry, and that the number of runs should be large enough that an atypical run has limited effect on the sample average [`txdot_number_runs`, `fhwa_microsim_2019`].

In this thesis, the run count is treated as a precision-control choice rather than a universal constant. A small number of runs may be enough when demand is stable and candidate plans are clearly separated. More runs are needed when congestion is saturated or when the best plans have similar mean performance. The implementation therefore keeps all paired per-run observations, so that convergence plots and confidence intervals can be inspected after the simulations and the run budget can be extended when the recommendation is still unstable.

2.4 Multiple-criteria decision analysis

Multiple-criteria decision analysis (MCDA) supports decisions where alternatives must be evaluated on several criteria that are not naturally reducible to one physical unit. The UK government’s MCDA manual describes this as a framework for appraising policy and project options when monetary valuation alone is insufficient. It emphasizes explicit criteria, explicit scores, explicit weights and transparent judgement [`mcda_manual`]. This is exactly the structure of deviation-plan evaluation: network delay, mean travel time, throughput feasibility and impacted-user burden are all relevant, but they do not express the same dimension of performance.

MCDA is not used here to hide the trade-off inside one unexplained number. Instead, it is used to make the decision rule auditable. The evaluator first removes infeasible or dominated alternatives, then ranks the remaining plans with an additive score and finally tests whether the ranking is sensitive to the chosen weights.

2.4.1 Pareto dominance and non-dominated plan sets

Pareto dominance provides a weight-free screening rule. A feasible plan a *Pareto-dominates* a feasible plan b if a is no worse than b on every criterion and strictly better on at least one. Grierson gives the same formal condition for Pareto-optimal solutions in a multi-criteria minimization problem [GRIERSON2008371].

The non-dominated plans form the Pareto front. Restricting scoring to this set has a practical benefit: no choice of non-negative weights can justify selecting a plan that is uniformly worse than another feasible plan. This is particularly useful in deviation planning because a plan can look attractive on one metric while being worse on all others once paired simulation results are considered.

2.4.2 Additive weighted scoring

Additive weighted scoring is a simple and widely used aggregation method in multi-criteria decision analysis. Each alternative is evaluated on several criteria, whose values are first transformed onto a common scale. The overall score of alternative k is then computed as

$$S_k = \sum_q w_q z_{kq}, \quad (1)$$

where z_{kq} is the scaled performance of alternative k on criterion q , and w_q is the weight assigned to that criterion. The weights are non-negative and usually sum to one.

This approach is valued for its transparency: the final ranking follows directly from the selected criteria and their weights [mcda_manual]. When all criteria are formulated as costs, lower scores indicate better alternatives. Its main limitation is that it allows compensation between criteria, meaning that poor performance on one criterion may be offset by good performance on another.

2.4.3 Weight sensitivity and robustness analysis

The main limitation of additive scoring is that weights encode value judgements. Sensitivity analysis is therefore part of the decision method, not an optional afterthought. Triantaphyllou and Sanchez similarly frame sensitivity analysis as a way to examine changes in criteria weights and alternative performance values for deterministic MCDM methods, including the weighted sum model [triantaphyllou_sensitivity_1997].

In this work, weight robustness is estimated by drawing many admissible weight vectors from the simplex and recomputing the ranking. The sampling distribution is $\text{Dirichlet}(1, \dots, 1)$, which is uniform over the weight simplex [stan_dirichlet_simplex]. The robustness score of a plan is the fraction of sampled weight vectors for which it ranks first. A high robustness score means that the recommendation does not depend strongly on the reference weight vector, whereas a low robustness score signals that the decision should be escalated to explicit stakeholder weighting.

3 Methodology

3.1 System overview and pipeline

The methodology follows the same order as the implemented workflow. The first step generates a synthetic, network-aware demand from the SUMO road network and the TAZ layer. This demand is written as `trips.xml`: every trip has a departure time, an origin edge and a destination edge, while the full path is left open. The closure is then translated into candidate deviation plans. In the Monte Carlo stage, the same trip demand is routed repeatedly with `duarouter`⁴, using randomized edge weights, before each deviation plan is applied and simulated in SUMO. The last step compares the plans using paired statistics and a multi-criteria decision rule. The pipeline is illustrated in **Figure ??**.

Module	Main role	Main outputs
<code>scenario_generator</code>	Generates synthetic OD demand and unrouted trip records	<code>trips.xml</code> , OD matrices
<code>deviation_plan_maker</code>	Converts the network into a graph and calculates the deviation plan(s)	JSON of candidate deviation plans
<code>monte_carlo_simulation</code>	Re-routes trip demand and simulates baseline and detour conditions over multiple runs	<code>run_#</code> folders, <code>summary.json</code>
<code>evaluator</code>	Paired statistical analysis and multi-criteria ranking	<code>evaluation.json</code> , <code>evaluation.md</code>

Table 1: Functional organization of the pipeline

3.2 Scenario generation

The scenario generator creates the demand used by the rest of the pipeline. In this work, the demand must satisfy four constraints:

1. **Network-aware:** Origins and destinations are sampled from actual SUMO road edges.
2. **Zone-aware:** Demand is expressed at the TAZ level.
3. **Temporally structured:** The daily profile includes morning and evening peaks.
4. **Directional:** Peak periods can concentrate trips from residential zones toward employment zones and back.

3.2.1 Module overview

The generation is split into three stages:

⁴<https://sumo.dlr.de/docs/CLI.html>

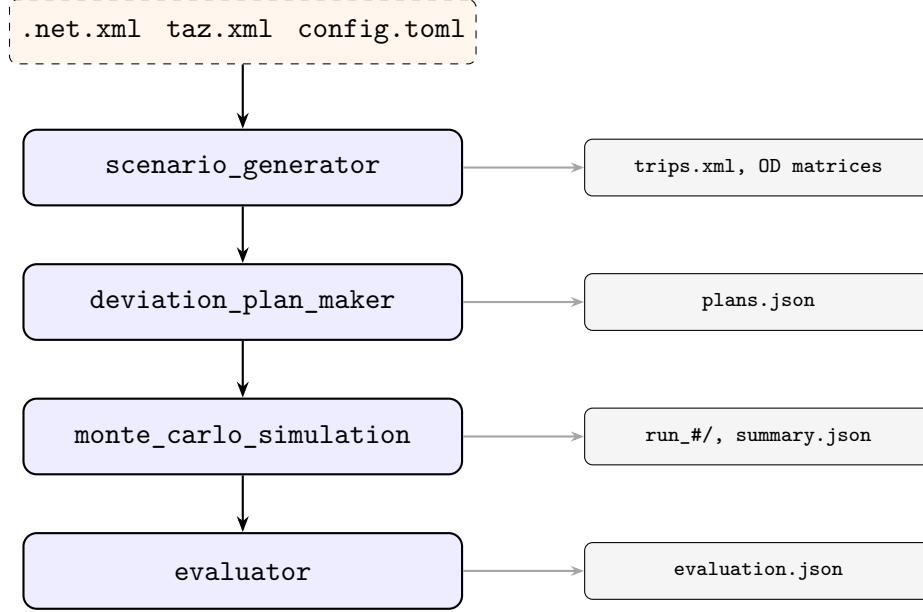


Figure 3: Four-module pipeline. Inputs (dashed box) flow top-to-bottom through the four modules. The scenario generator writes unrouted trips; explicit route files are produced later inside each Monte Carlo iteration by `duarouter`.

1. **Stage 1 - Demand generation:** construct 24 hourly Origin–Destination (OD) matrices using a gravity model with an optional directional commute overlay.
2. **Stage 2 - Route library:** build a small set of candidate routes for each active OD pair using Yen’s k -shortest paths algorithm [`yen_algorithm`].
3. **Stage 3 - Trip instantiation:** assign a departure time and valid endpoint edges to every trip. The full route is assigned later, inside the Monte Carlo stage.

3.2.2 Stage 1 : Demand generation

3.2.2.1 Traffic Analysis Zones

The spatial unit of the model is the *Traffic Analysis Zone* (TAZ), a standard construct in transport demand modelling that aggregates individual trip origins and destinations into discrete geographic areas. TAZs in SUMO are defined as polygons that can be associated with the road network. Each TAZ is defined by the set of SUMO road edges it contains.

A scalar *zone mass* m_i is assigned to each TAZ as the capacity of its drivable edge lengths :

$$m_i = \sum_{e \in \mathcal{E}_i} \ell(e) \cdot n(e) \quad (2)$$

where $\ell(e)$ is the length of edge e in meters and $n(e)$ is its lane count.

Equation ?? is an approximation, used because no calibrated socio-economic productions or attractions are available for this synthetic setting. In a classical four-step transport model, those quantities would be estimated from socio-economic data [`modelling_transport`]. Here, lane kilometres⁵ are used as a practical surrogate: they are directly available from

⁵A lane kilometre is one kilometre of road with one lane.

the network and major road corridors generally carry more traffic than minor links. If observed productions and attractions become available, they should replace this proxy.

3.2.2.2 Temporal Demand Profile

The total daily demand N is distributed across the 24 hours of the simulation using a *two-Gaussian profile*:

$$p_h = \frac{a_m g(h; \mu_m, \sigma) + a_e g(h; \mu_e, \sigma)}{\sum_{h'=0}^{23} [a_m g(h'; \mu_m, \sigma) + a_e g(h'; \mu_e, \sigma)]}, \quad (3)$$

where

$$g(h; \mu, \sigma) = \exp\left(-\frac{(h - \mu)^2}{2\sigma^2}\right), \quad (4)$$

μ_m and μ_e are the morning and evening peak hours respectively and σ controls the temporal spread. The amplitudes a_m and a_e allow the two peaks to differ in magnitude. The default configuration uses $a_e > a_m$, representing a slightly larger afternoon peak, which is common in urban networks [gaussian_book].

The continuous profile p_h is converted into hourly trip counts n_h with the largest-remainder method. This keeps the total exactly equal to N while staying as close as possible to the continuous proportions.

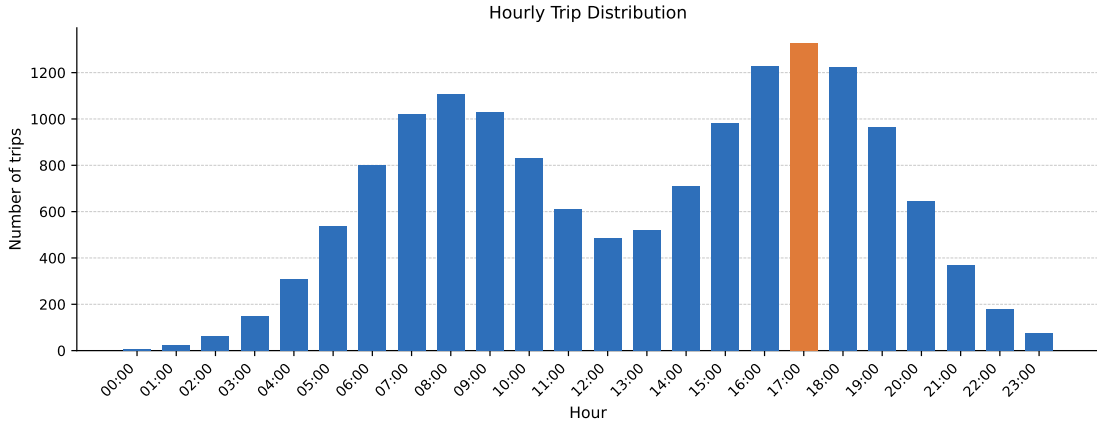


Figure 4: Hourly trip distribution produced by the two-Gaussian profile (Equation ??) for the test scenario. The orange bar marks the evening peak, which is slightly larger than the morning peak ($a_e > a_m$).

3.2.2.3 Gravity OD model

The spatial distribution of trips across TAZ pairs is governed by a *singly-constrained gravity model* [gravity_model]. The **unnormalized** weight of the OD pair (i, j) is :

$$w_{ij} = m_i \cdot m_j \cdot \exp(-\beta c_{ij}), \quad i \neq j, \quad (5)$$

where c_{ij} is the travel-cost impedance between zones i and j (shortest path length in meters) and $\beta \geq 0$ is the distance decay parameter⁶.

⁶A larger β means trips decay faster with distance/cost (people are more reluctant to travel far), a smaller β means the distribution is more spatially spread out. Wilson discusses this in the context of

Pairs with infinite or negative impedance receive zero weight. The normalized *OD share matrix* is then:

$$S_{ij} = \underbrace{\frac{m_i}{\sum_k m_k}}_{\text{origin share } O_i} \cdot \underbrace{\frac{m_j \exp(-\beta c_{ij})}{\sum_\ell m_\ell \exp(-\beta c_{i\ell})}}_{\text{conditional destination probability } p(j|i)}, \quad (6)$$

so that, by construction, $\sum_j S_{ij} = m_i / \sum_k m_k$ and $\sum_{ij} S_{ij} = 1$.

For each hour h , the share matrix is scaled to the corresponding integer total n_h with the largest-remainder method, giving an exact integer OD matrix T^h with $\sum_{ij} T_{ij}^h = n_h$.

The model is *singly-constrained on the production side*. Although the weight $w_{ij} = m_i m_j \exp(-\beta c_{ij})$ depends symmetrically on origin and destination masses, the normalisation in Equation ?? enforces the row-sum constraint

$$\sum_j S_{ij} = \frac{m_i}{\sum_k m_k}$$

while leaving the column sums $\sum_i S_{ij}$ unconstrained. In Wilson’s balancing-factor formulation [**modelling_transport**], this corresponds to enforcing only the origin-side factor A_i and fixing the destination-side factor $B_j \equiv 1$. A doubly-constrained model would additionally enforce observed zonal attractions and would require both balancing factors to be recovered by iterative proportional fitting, zonal attractions are not observed in the present synthetic setting, so enforcing them would add parameters that cannot be identified from the available data. For this reason, the singly-constrained formulation is used here: it relies only on the information present in the network (zone masses and impedances) and lets destination totals emerge from the distance-decay term $\exp(-\beta c_{ij})$.

3.2.2.4 Commute Overlay (Optional)

When residential weights r_i and employment weights e_i are available, the hourly OD shares can be tilted toward commuting patterns. Two additional gravity matrices are built: one for the morning peak, from residential zones to employment zones and one for the evening peak, in the reverse direction:

$$w_{ij}^{\text{AM}} = m_i m_j f(c_{ij}) r_i e_j, \quad (7)$$

$$w_{ij}^{\text{PM}} = m_i m_j f(c_{ij}) e_i r_j, \quad (8)$$

where $f(c_{ij}) = \exp(-\beta_c c_{ij})$ discounts longer or costlier trips. In the AM matrix, r_i and e_j increase flows leaving residential zones and arriving at employment zones. In the PM matrix, the same logic is reversed.

The two directional matrices are blended with the base OD share matrix through a time-varying mixture. For each hour h , the blended share is:

$$S_{ij}^h \propto S_{ij}^{\text{base}} + \lambda g(h; \mu_m, \sigma) S_{ij}^{\text{AM}} + \lambda g(h; \mu_e, \sigma) S_{ij}^{\text{PM}}, \quad (9)$$

where $g(h; \mu, \sigma)$ is a Gaussian kernel centred on hour μ with spread σ . The parameter λ (**peak_weight**) controls how strongly the commute pattern modifies the background gravity model. Setting $\lambda = 0$ recovers the base model. After blending, the shares are normalised before being converted into integer trips.

calibrating the model to observed data.[**gravity_model**]

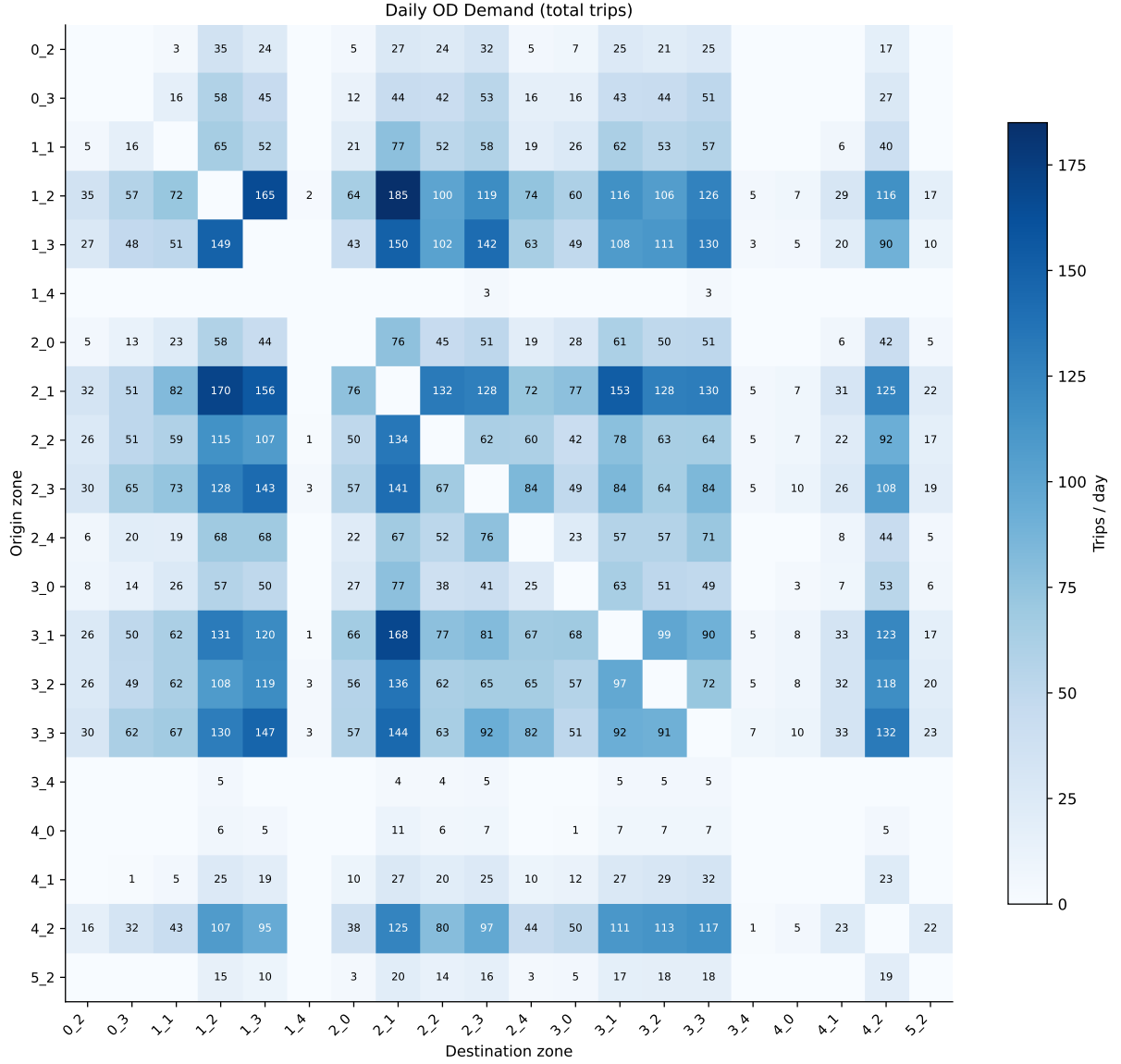


Figure 5: Daily OD demand matrix $\sum_h T^h$ for the test scenario (20 TAZs, 10 000 daily trips). Rows are origin zones, columns are destination zones. The cell colour and annotation encode the total number of trips. The diagonal is zero by construction (no intra demand). Dense clusters correspond to adjacent zones with high lane-kilometer mass.

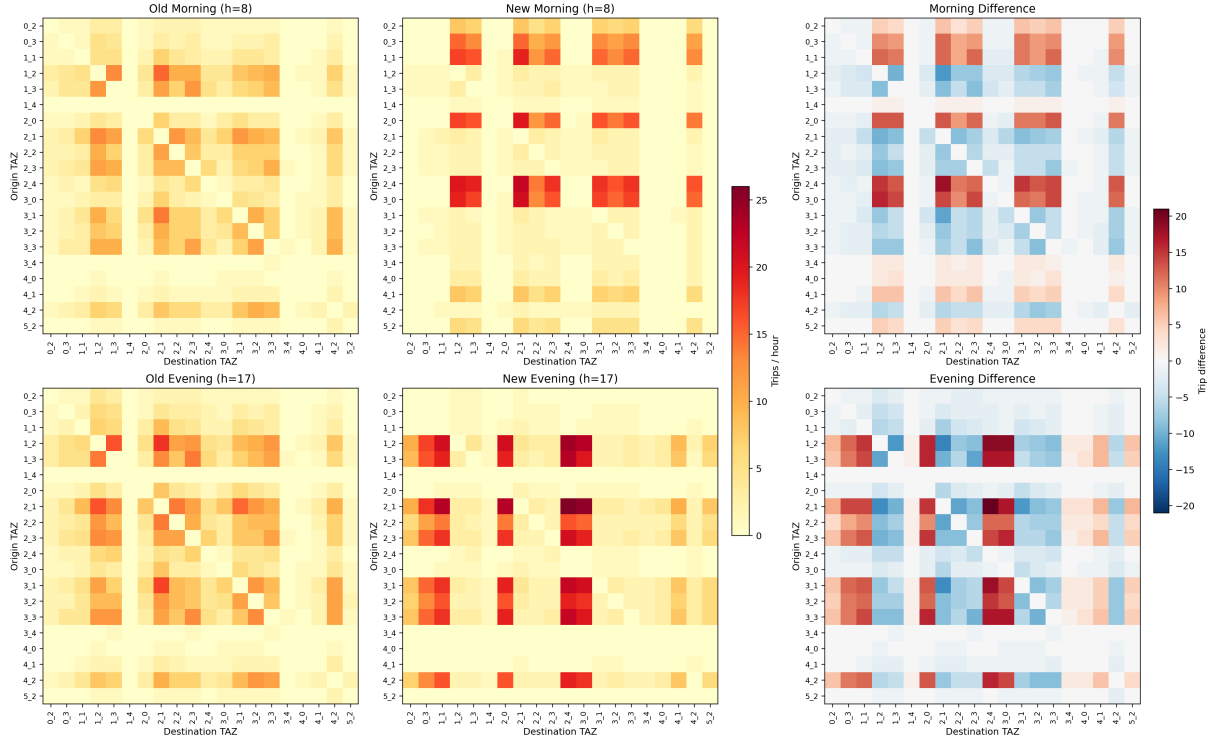


Figure 6: Effect of the commute overlay on the hourly OD matrices. Left column: base gravity model ($\lambda = 0$). Centre column: model with commute overlay ($\lambda > 0$). Right column: difference (centre – left). Top row: morning peak hour ($h = 8$). Bottom row: evening peak hour ($h = 17$). Red cells in the difference maps indicate flows amplified by the overlay, blue cells indicate flows attenuated. The AM overlay concentrates demand on residential→employment corridors, the PM overlay reverses this pattern.

3.2.3 Stage 2 : Route library

3.2.3.1 Edge expanded network

Routing for the scenario generator uses an *edge-expanded* graph: SUMO road edges are the graph nodes and a directed arc connects edge u to edge v when SUMO allows the movement from u to v . The weight of this arc is the length of v , so a path cost corresponds to the sum of the edge lengths in the route.

This representation is convenient for two reasons. First, the generator has to sample SUMO edge endpoints and `duarouter` later expands those endpoints into complete routes. Staying at edge level keeps the generated demand close to SUMO’s own trip format. Second, turning feasibility is already encoded by SUMO connections: if an arc exists, the turn is allowed. Internal junction edges⁷ and edges that do not permit private-vehicle access are excluded when the graph is built.

3.2.3.2 Impedance Estimation

The gravity model needs a TAZ-to-TAZ impedance matrix. Computing exact all-pairs shortest paths would be expensive and would add precision that is not needed at this stage. Instead, for each directed zone pair (i, j) , the generator samples a small number of origin-edge and destination-edge combinations from the two zones. The lowest observed shortest-path cost is used as the representative impedance. This is an approximation, but it gives the gravity model the spatial signal it needs: nearby and well-connected zones receive lower costs than distant or poorly connected ones.

3.2.3.3 k -Shortest Candidate Routes

For each active OD pair (i.e. $T_{ij}^h > 0$ for at least one hour h), a set of at most k candidate routes is computed as follows:

1. Sample s concrete (origin-edge, destination-edge) pairs, where edges are drawn proportionally to zone mass (Equation (??)).
2. For each pair, enumerate up to k shortest simple paths using Yen’s algorithm [`yen_algorithm`]
3. Deduplicate across all samples by edge sequence, if the same sequence appears more than once, retain the lowest-cost instance.
4. Return at most k candidates sorted by cost.

3.2.4 Stage 3 : Trip Instantiation

Once the hourly OD matrices and the route library are available, every integer trip becomes a SUMO `trip` record. Two quantities are sampled: the departure time and a feasible candidate path. Only the first and last edges of that path are kept, because the final route is intentionally left to `duarouter` in the Monte Carlo stage.

⁷SUMO ids beginning with :

3.2.4.1 Departure Time

Trips within each one-hour interval $[b, b + 3600)$ are assumed to depart uniformly over that interval. For an OD cell containing n trips, departure times are drawn independently as:

$$t_1, \dots, t_n \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}(b, b + 3600). \quad (10)$$

With only hourly totals available, this is the least committed assumption: there is no information that would justify placing trips at a more precise time within the hour.

3.2.4.2 Endpoint Selection

For an OD pair with candidate route set $\mathcal{R}_{ij}$, each trip first selects one candidate path uniformly at random:

$$P(r \mid \mathcal{R}) = \frac{1}{|\mathcal{R}|}, \quad r \in \mathcal{R}. \quad (11)$$

Only the first and last edges of the selected candidate are exported:

$$\text{from} = r_1, \quad \text{to} = r_{|r|}. \quad (12)$$

This produces an unrouted `trips.xml` file. In this mode, the route library is used only to choose feasible endpoints, it is not a fixed route assignment. The full path choice is delegated to `duarouter` inside the Monte Carlo loop, where edge costs are randomized independently for each run.

The departure-time and endpoint draws use one seeded random-number generator. As a result, the generated `trips.xml` is reproducible. This matters because the Monte Carlo experiment is meant to vary routing and simulation seeds, not the underlying trip population.

3.3 Deviation plan computation

After demand generation, the next step is to define how the network can bypass the closure. The `deviation_plan_maker` takes the SUMO network and a user-selected closure as input, then returns candidate detour paths around the disrupted segment. These plans are not evaluated at this stage, they are passed unchanged to the Monte Carlo simulation.

Formally, let $G = (V, E, \ell)$ be a weighted directed graph in which V is the set of junctions, E the set of road edges and $\ell : E \rightarrow \mathbb{R}_{>0}$ an edge length function. Given a set of closed edges $E_c \subset E$ and a blocked set $B \supseteq E_c$, a *deviation plan* is defined as a tuple

$$\mathcal{D} = (s, t, P, \ell(P), d), \quad (13)$$

where $s \in V$ is the *origin* of this particular plan (the junction at which the detour begins) and $t \in V$ is the detour target, while $P = (e_1, \dots, e_m)$ is a loopless path in $G' = (V, E \setminus B)$ from s to t and $\ell(P) = \sum_{i=1}^m \ell(e_i)$ is its total geometric length in meters. The integer $d \in \mathbb{N}_0$ is the *BFS depth* of s relative to the *effective detour source* returned by the endpoint computation of Section ???. In the common case this effective source coincides with the upstream junction of the first closed edge $s_0 = \text{from}(e_1^c)$. If that junction has no usable alternative exit, the endpoint computation moves the source further upstream and adds

the traversed prefix edges to the blocked set B . In the upstream method, the origin s can differ from one plan to another: the effective source, t and B are shared, but individual plans may start further upstream. When $d = 0$, the plan starts at the effective source and acts as the shallowest fallback. Plans with $d > 0$ start further upstream and are tried first during multi-plan route rewriting.

3.3.1 Junction-level graph representation

The `deviation_plan_maker` operates on a *junction-level* directed graph, in contrast to the edge-expanded representation used by the scenario generator (Section ??). Here nodes correspond to SUMO junctions and arcs correspond to road edges: an arc (u, v) exists for every edge whose `from` attribute is junction u and whose `to` attribute is junction v . Arc weights default to the geometric length of the edge in meters. An alternative weight can be loaded from a SUMO `edgedata.xml` file, in which case values are averaged across all measurement intervals and the resulting mean is stored as the weight, if the field encodes speed rather than travel time, the reciprocal is stored instead so that shorter weighted distances correspond to faster edges.

The graph is built directly from the `.net.xml` file. The implementation keeps three dictionaries (`edges`, `outgoing` and `incoming`) that provide $O(1)$ lookup of any edge record and $O(\deg(v))$ enumeration of a node's neighbours⁸. These structures are sufficient for the routing operations used in this module.

3.3.2 Detour endpoint computation

Given a sequence of closed edges $E_c = (e_1, \dots, e_n)$, the natural candidate detour source is the upstream junction of e_1 and the candidate target is the downstream junction of e_n . However, if the source junction has only one outgoing edge (the first closed edge itself), no detour can diverge from it, the same problem arises symmetrically at the target if it has only one incoming edge.

To handle this, the algorithm walks upstream from the source and downstream from the target until it reaches junctions with at least two outgoing (resp. incoming) edges. Every intermediate edge encountered during these walks is added to the blocked set, so the router cannot reuse it as a shortcut. **Figure ??** illustrates the procedure on a small graph in which the natural endpoints s_0 and t_0 are both dead ends, and the walks promote s and t as the actual detour endpoints.

3.3.3 Candidate generation

Two candidate-generation methods are implemented. Both start from the same endpoint computation, but they use it differently. Yen's method keeps one detour source s and enumerates the k best loopless paths from s to t , all of its plans have depth $d = 0$. The upstream method starts from the same effective source, then searches backwards up to $d_{\max}$ hops. Each upstream node becomes a possible plan origin and the shortest path

⁸Here, $\deg(v)$ denotes the number of outgoing or incoming edges adjacent to node v , depending on the direction of the enumeration. Thus, neighbour enumeration is linear in the local branching factor of the queried junction, not in the total number of edges in the network.

Algorithm 1 FINDDETOURENDPOINTS (high-level idea)

Require: closed edge sequence E_c , graph G

Ensure: source s , target t , extended blocked set B

- 1: start with s and t at the upstream and downstream ends of the closure
 - 2: initialise B with the closed edges
 - 3: **while** s has only one way out (the closure itself) **do**
 - 4: step one junction further upstream and add the traversed edge to B
 - 5: **end while**
 - 6: **while** t has only one way in **do**
 - 7: step one junction further downstream and add the traversed edge to B
 - 8: **end while**
 - 9: **return** s, t, B
-

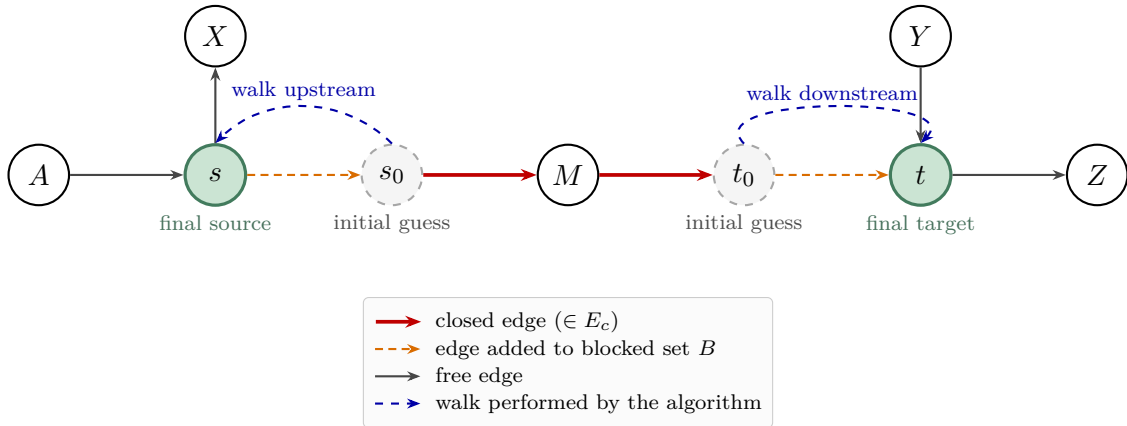


Figure 7: Detour endpoint computation on an illustrative graph. The closure $E_c = (s_0 \rightarrow M, M \rightarrow t_0)$ is shown in red. Because s_0 has only one outgoing edge (the closure itself) and t_0 has only one incoming edge, neither can serve as a detour endpoint. The algorithm walks one junction upstream from s_0 to s (which has the additional outgoing edge $s \rightarrow X$) and one junction downstream from t_0 to t (which has the additional incoming edge $Y \rightarrow t$). The two traversed edges (orange, dashed) are appended to the blocked set B so they cannot be reused by the router as shortcuts around the closure.

from that origin to t is retained if it exists. Yen’s method is the default in the end-to-end pipeline. The upstream method is used when the experiment explicitly needs detours that begin before the closure point.

3.3.3.1 Yen’s k -shortest loopless paths.

With source s , target t and blocked set B fixed, the first method calls Yen’s k -shortest loopless paths algorithm [**yen_algorithm**] to enumerate up to k alternative routes from s to t that avoid every edge in B .

As described in **Algorithm ??**, Yen’s algorithm extends Dijkstra’s shortest-path search by iteratively generating *spur paths*. Let P_1 be the shortest path returned by Dijkstra. At each subsequent iteration i , the algorithm considers every prefix of the previously accepted path P_{i-1} as a potential root segment. For each root, it temporarily blocks the outgoing arc that was used by any already-accepted path sharing that root, then runs Dijkstra from the spur node (the last node of the root) to t . The resulting spur path is concatenated with the root to form a complete candidate. All candidates are stored in a min-heap keyed by total cost and the minimum is promoted to P_i . A global signature set prevents the same edge sequence from appearing twice. Heap entries carry a monotonically increasing tie-breaking counter, so that two paths with identical cost are ordered by insertion time rather than by a potentially fragile lexicographic comparison of node lists.

Algorithm 2 YENKSHORTESTPATHS (high-level idea)

Require: graph G , source s , target t , integer $k \geq 1$, blocked set B

Ensure: list $\mathcal{P}$ of up to k diverse shortest paths from s to t avoiding B

```

1:  $P_1 \leftarrow$  shortest path from  $s$  to  $t$  avoiding  $B$ 
2:  $\mathcal{P} \leftarrow [P_1]$ 
3: for  $i \leftarrow 2$  to  $k$  do
4:   candidates  $\leftarrow \emptyset$ 
5:   for each prefix of the previously accepted path  $P_{i-1}$  do
6:     temporarily block the next arc taken by any already-accepted path that shares
       this prefix
7:     find the shortest path from the prefix’s endpoint to  $t$ 
8:     store prefix + new path as a candidate
9:   end for
10:  pick the cheapest unseen candidate as  $P_i$  and append it to  $\mathcal{P}$ 
11:  stop if no candidate remains
12: end for
13: return  $\mathcal{P}$ 
```

3.3.3.2 Upstream search.

The upstream method is based on the fact that vehicles approaching a closure may arrive from several directions, not from one junction only. A reverse BFS from s records every reachable upstream node u and its depth $d(u)$ up to $d_{\max}$. The source s itself is kept at depth 0, so the method still contains a fallback. For each discovered origin, Dijkstra is run from that origin to t while avoiding the blocked set B .

The returned plans are sorted by depth descending and then by total length. This ordering is used later by the route rewriter: it tries the most upstream applicable plan first, then falls back to shallower ones.

Algorithm 3 UPSTREAMDEVIATIONPLANS (high-level idea)

Require: graph G , endpoints s, t , blocked set B , max depth $d_{\max}$

Ensure: deviation plans sorted by depth (desc), then length (asc)

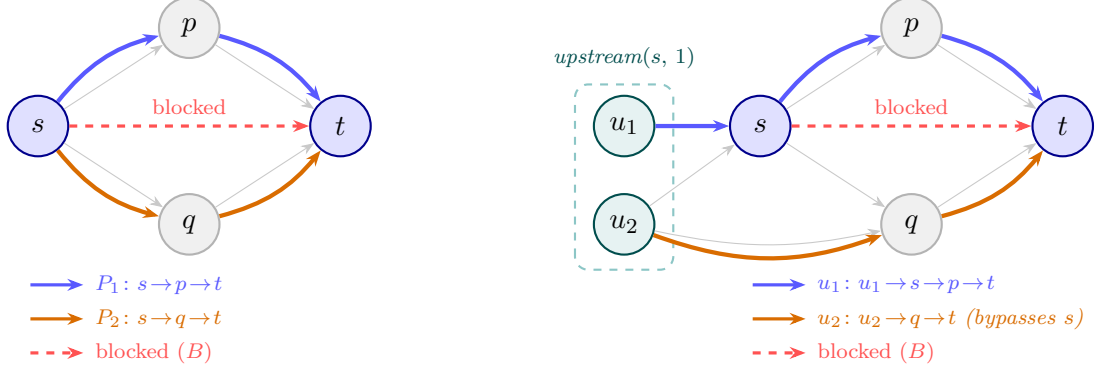
```

1: reverse-BFS from  $s$  up to  $d_{\max}$  hops, tagging each reachable node  $u$  with its depth  $d(u)$ 
2:  $\mathcal{P} \leftarrow []$ 
3: for each upstream origin  $u$  found (including  $s$  at depth 0) do
4:   compute the shortest path  $\pi$  from  $u$  to  $t$  avoiding  $B$ 
5:   if  $\pi$  exists and is not a duplicate then
6:     save  $\mathcal{D}(\pi, d(u))$  as a candidate plan in  $\mathcal{P}$ 
7:   end if
8: end for
9: sort  $\mathcal{P}$ : deeper origins first, shorter paths first within each depth
10: return  $\mathcal{P}$ 

```

where $\mathcal{D}(\pi, d)$ denotes the deviation plan wrapping path π with BFS depth d .

Figure ?? illustrates the two methods on the same toy network: nodes s and t are the detour endpoints, p and q are intermediate junctions and the direct edge $s \rightarrow t$ belongs to the blocked set B .



(a) Yen's k -shortest paths ($k = 2$)

(b) Upstream multi-origin ($d = 1$)

Figure 8: Comparison of the two candidate generation methods on a toy network. In both cases the edge $s \rightarrow t$ is blocked (red dashed). (a) Yen's algorithm produces $k = 2$ loopless paths, both originating from s : P_1 takes the upper corridor through p and P_2 the lower corridor through q . (b) The upstream BFS at depth $d = 1$ identifies two origins $\{u_1, u_2\}$ (teal). u_1 routes through s and then takes the upper corridor, while u_2 holds a direct connection to q , so its optimal path bypasses s entirely, a candidate that Yen's method, being anchored at s , would never generate.

Each accepted path is wrapped in a **DeviationPlan** object. The object records the plan identifier, its rank within the k -shortest list, the ordered sequences of junction and edge identifiers and the total geometric length in meters (computed by summing the **length_m** attribute of every edge on the path).

No post-filtering is applied at this point. Plans are not removed because of a maximum detour-length ratio and near-duplicates are not removed by edge-set overlap. This keeps the planning stage simple and avoids discarding a route before it has been tested in simulation. Two routes can share most of their edges and still behave differently if they diverge near a bottleneck or pass through a different sequence of signalised intersections. The Monte Carlo evaluation is used later to separate useful candidates from poor ones.

3.3.4 Route rewriting

Detours are applied by rewriting route files before SUMO starts, rather than by using SUMO's dynamic *rerouters*⁹. This keeps the experiment static and inspectable: **duarouter** already provides route-choice variation through randomized edge weights and the final route file can be checked before the simulation.

After each Monte Carlo call to **duarouter**, the rewriter creates one **routes_{plan_id}.xml** file per candidate plan. Only vehicles that traverse a closed edge are modified. For each affected vehicle, candidate plans are tried from deepest to shallowest and the first plan whose origin appears upstream of the closure in that vehicle's route is selected (Algorithm ??). Yen plans have only the evaluated depth-0 candidate, upstream plans also carry the depth-0 fallback.

The selected plan $\mathcal{D}^*$ is inserted by replacing the route segment from $\mathcal{D}^*.s$ to the last closed edge with the detour edge sequence $\mathcal{D}^*.P$. This is a splice in the vehicle's edge list, illustrated in Figure ??.

Algorithm 4 DEPTHAWAREWRITE (high-level idea)

Require: vehicle route R , candidate plans $\mathcal{D}$ sorted by depth (desc), closed edge set B_c

Ensure: rewritten route R'

- 1: **if** R does not traverse any edge in B_c **then**
 - 2: **return** R *// route unaffected by the closure*
 - 3: **end if**
 - 4: pick the deepest plan $\mathcal{D}^*$ whose origin s lies on R upstream of the closure *// shallower plans provide deterministic fallback*
 - 5: splice $\mathcal{D}^*.P$ into R , replacing the segment that runs from the edge departing $\mathcal{D}^*.s$ up to the last closed edge
 - 6: **return** the spliced route
-

The baseline route file is left unchanged, each candidate is evaluated through its own rewritten copy.

3.4 Monte Carlo Simulation

The Monte Carlo stage turns the fixed demand and the candidate detours into repeated SUMO observations. In this stage, the demand stays fixed while the route assignment changes. The scenario generator gives the simulation an unrouted **trips.xml** file and each Monte Carlo iteration routes that same trip set again with **duarouter**, a different seed

⁹<https://sumo.dlr.de/docs/Simulation/Rerouter.html>

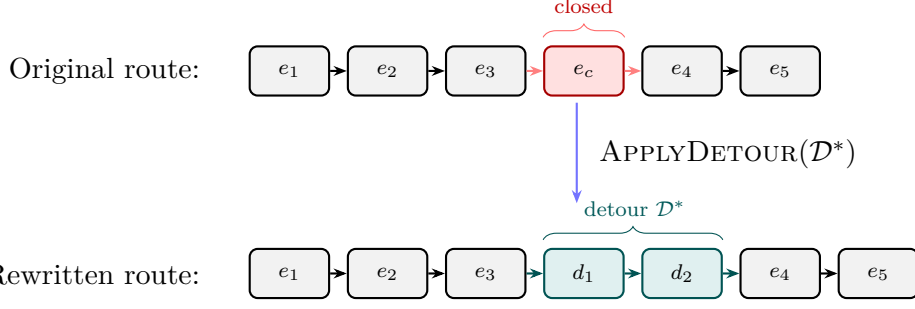


Figure 9: Route rewriting for one affected vehicle. The closed edge e_c (red) is replaced by the selected detour sequence (d_1, d_2) (teal), while the unaffected prefix and suffix are preserved.

and randomized edge weights.

3.4.1 Experimental design

The experiment uses a **paired stochastic-routing design**. The trip population is fixed, while the route realization changes from run to run. Let $\mathcal{T}$ be the `trips.xml` file produced by the scenario generator. For Monte Carlo iteration r , the simulation first computes the baseline route file

$$\mathcal{R}_0^{(r)} = \text{DUAROUTER}(\mathcal{T}, G, r, \alpha), \quad (14)$$

where G is the SUMO network and α is the upper bound passed to `-weights.random-factor`. With this option, `duarouter` perturbs edge costs by drawing multipliers in $[1, \alpha]$. Different iterations can produce different paths for the same trip endpoints.

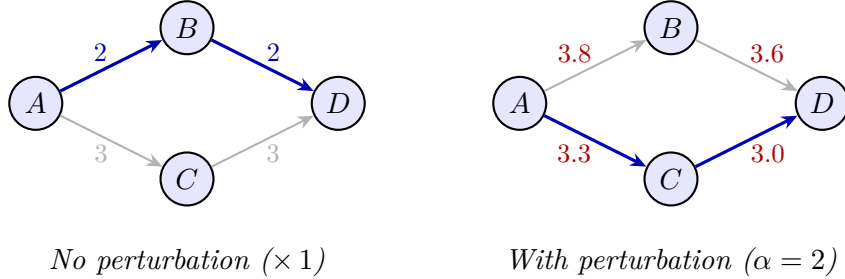


Figure 10: Effect of `-weights.random-factor` on route choice. Both graphs share the same network and origin–destination pair $A \rightarrow D$. **Left:** nominal edge costs, the router selects $A \rightarrow B \rightarrow D$ (cost 4) over $A \rightarrow C \rightarrow D$ (cost 6). **Right:** each cost is multiplied by a random factor drawn from $[1, \alpha]$, the top path becomes costlier (7.4) than the bottom path (6.3), so the router now chooses $A \rightarrow C \rightarrow D$. Across Monte Carlo iterations, different draws of these multipliers yield different route realizations for the same trip set.

For every candidate deviation plan $\mathcal{D}_k$, the deterministic rewriter is then applied to that same baseline route file. The baseline and all plan conditions are paired within the same run: they share the same trip set, the same route realization before applying the closure treatment and the same SUMO seed.

For each iteration r , the raw time-loss difference

$$\delta_k^{(r)} = D_k^{(r)} - D_0^{(r)} \quad (15)$$

measures the effect of plan $\mathcal{D}_k$ relative to the no-closure baseline under the same route-choice and SUMO random seed. The estimator

$$\bar{\delta}_k = \frac{1}{R} \sum_{r=1}^R \delta_k^{(r)} \quad (16)$$

compares plans after much of the run-level noise shared by the baseline and plan conditions has been removed.

3.4.2 Per-run routing and detour application

There is no plan-specific route pre-computation outside the Monte Carlo loop. The only fixed demand input is `trips.xml`. In each iteration, `duarouter` writes `run_#/routes.xml`. This is the baseline route realization for that seed. The route rewriter then creates the `routes_{plan_id}.xml` files in the same run directory.

After each plan file is written, the code checks that the closed edge no longer appears in any rewritten route. It also compares the baseline and plan files vehicle by vehicle to count how many routes changed. These statistics are stored per run, since the affected vehicles can change when `duarouter` produces a different baseline route set.

3.4.3 Iteration structure

Each Monte Carlo iteration consists of routing, deterministic route rewriting, SUMO execution and metric extraction. Algorithm ?? states the procedure formally, Figure ?? summarizes the workflow.

Algorithm 5 MONTECARLOITERATION

Require: network file G , trip file $\mathcal{T}$, closed edge e_c , candidate plans $\{\mathcal{D}_1, \dots, \mathcal{D}_K\}$, edge-endpoint map $\mathcal{E}$, run index r , simulation window $[b, e]$, random factor α

Ensure: per-condition metric dictionary for this iteration

```

1:  $\mathcal{R}_0^{(r)} \leftarrow \text{DUAROUTER}(\mathcal{T}, G, r, \alpha)$  ▷ baseline route realization
2:  $M_0 \leftarrow \text{RUNSUMO}(G, \mathcal{R}_0^{(r)}, r, b, e)$ 
3: results[baseline]  $\leftarrow M_0$ 
4: for  $k \leftarrow 1$  to  $K$  do
5:    $\mathcal{R}_k^{(r)} \leftarrow \text{APPLYDETOURTOROUTES}(\mathcal{R}_0^{(r)}, \mathcal{D}_k, e_c, \mathcal{E})$ 
6:   validate that  $\mathcal{R}_k^{(r)}$  contains no closed edge  $e_c$ 
7:    $s_k^{(r)} \leftarrow \text{COUNTREROUTED}(\mathcal{R}_0^{(r)}, \mathcal{R}_k^{(r)})$ 
8:    $M_k \leftarrow \text{RUNSUMO}(G, \mathcal{R}_k^{(r)}, r, b, e)$ 
9:   results[ $\mathcal{D}_k$ ]  $\leftarrow M_k \cup s_k^{(r)}$ 
10: end for
11: return results
```

`DUAROUTER` invokes `duarouter` with the fixed trip file, `-weights.random-factor` and the run seed. `RUNSUMO` writes a minimal SUMO configuration file that specifies the

network, the route file, the simulation window $[b, e]$ and the same seed r , invokes `sumo` as a subprocess and parses the resulting `tripinfo.xml` output into the metrics of Section ???. Each condition runs in its own subdirectory (`run_r/baseline/`, `run_r/plan_k/`), keeping outputs isolated.

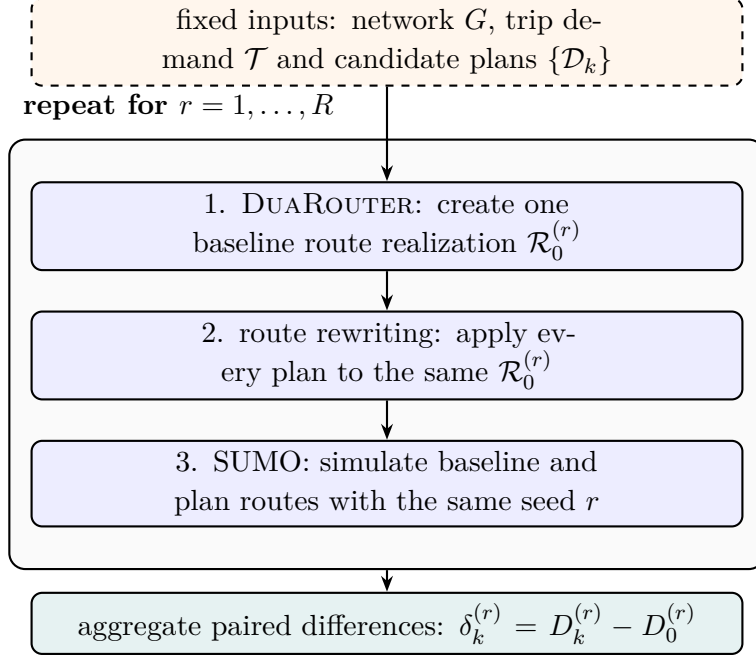


Figure 11: Monte Carlo protocol. The demand and candidate plans are fixed, each seed produces one baseline route realization, applies all detours to that realization and simulates paired baseline-plan conditions.

3.4.4 Performance metrics

Each `tripinfo.xml` file is parsed into a metric vector $M_c^{(r)}$ for condition c and run r . SUMO is configured with `tripinfo-output.write-unfinished`, so completed trips can be separated from unfinished or vaporized records. Aggregate performance is computed only on completed trips. The unfinished count is kept as a diagnostic.

- **Total time-loss delay D** (vehicle-hours): the sum of SUMO `timeLoss` over completed trips, converted from seconds to hours,

$$D = \frac{1}{3600} \sum_{v \in \mathcal{V}_{\text{done}}} \lambda_v, \quad (17)$$

where λ_v is the vehicle’s SUMO `timeLoss`. This is the excess travel time relative to SUMO’s ideal traversal of the driven route, so it captures both stopped waiting and moving slowdowns.

- **Mean travel time $\bar{\tau}$** (minutes): the arithmetic mean of completed trip durations,

$$\bar{\tau} = \frac{1}{60 |\mathcal{V}_{\text{done}}|} \sum_{v \in \mathcal{V}_{\text{done}}} \tau_v, \quad (18)$$

where τ_v is the door-to-door duration of trip v in seconds.

- **Completed and unfinished trips:** the number of vehicles that arrived within the simulation window and the number of records that did not. A high unfinished count flags a simulation window, demand, or topology problem that should be inspected before interpreting a plan.
- **Rerouted vehicles:** for each plan and run, the number and percentage of vehicles whose route sequence differs from the run’s baseline route file. This value is computed after `duarouter`, so it can vary across Monte Carlo iterations.

For plan comparison, the evaluator computes paired differences such as $\delta_k^{(r)}$ in Equation (??) run by run. These raw paired observations are preserved in the per-run `results.json` files and are not replaced by only top-level averages.

3.4.5 Convergence and number of runs

The number of runs is fixed by the user. If more precision is needed, the same output directory can be extended using a resume mechanism, which continues from the next missing run identifier.

3.5 Evaluation and decision support

The last step is the evaluation of the simulated plans. At this point no new traffic is generated and no additional SUMO simulations are run. The task is to take the repeated outputs from the Monte Carlo experiment and compare the candidate detours in a way that is fair, reproducible and useful for the final discussion. The key point is to keep the paired structure of the experiment: for each Monte Carlo seed, the evaluator has one baseline result and one result for each candidate deviation plan. Each plan is compared against the baseline from the same run, not against an unrelated average.

In practice, the evaluation is organised around three questions:

- How much each plan changes the network-level performance compared with the baseline.
- Whether the plan remains feasible, in particular whether it avoids reducing the number of completed trips.
- Among the feasible plans, which alternatives still look reasonable after Pareto screening and a sensitivity check on the decision weights.

3.5.1 Choice of evaluator metrics

The evaluator does not try to prove that one detour is universally best. It separates the comparison into quantities that answer different planning questions:

- **Total time-loss delay** measures the network-level congestion burden created or removed by a plan. It is the primary efficiency metric because it captures both queues and slow movement relative to the vehicle’s ideal route traversal.
- **Mean travel time** measures the average trip burden for completed vehicles. It complements time-loss delay because a plan can reduce congestion on the chosen route while still making paths longer, or conversely add local congestion while shortening

many trips.

- **Completed and unfinished trips** are used as a feasibility diagnostic. A plan must not appear good merely because more vehicles fail to finish within the simulation window and are therefore absent from completed-trip averages.
- **Fraction of impacted users delayed** focuses on the vehicles whose route actually changed. Network averages can hide a plan that performs well overall but imposes most of the cost on the directly affected users.

These metrics are also practical: they are available for every Monte Carlo run from SUMO's `tripinfo.xml` output and from the route comparison step, so the same definitions can be applied consistently to every candidate plan. The statistical tests reported by the evaluator are diagnostic checks on the paired effects, they are not the ranking rule. The final decision layer uses effect sizes, feasibility, Pareto dominance and weight sensitivity.

3.5.2 Comparable run set

The first operation is to identify the runs that can actually be compared. The evaluator scans the Monte Carlo output directory for `run_#/results.json` files and keeps a run only when it contains the baseline and all candidate plans listed in `summary.json`. Runs with missing plan outputs are left out of the paired analysis, but they are still recorded in the metadata so that the exclusion is visible.

This step avoids comparing one plan over more replications than another. After the filtering, every paired statistic is computed on the same run identifiers for every plan. If no complete run is available, the evaluator stops instead of producing a ranking based on incomplete evidence.

3.5.3 Paired network-level statistics

For every plan $\mathcal{D}_k$ and every retained Monte Carlo run r , the basic quantity is the difference between the plan and the baseline from the same run. For the primary network-level metric, this gives

$$\Delta D_k^{(r)} = D_k^{(r)} - D_0^{(r)}, \quad (19)$$

where $D_0^{(r)}$ is the baseline time-loss delay from run r and $D_k^{(r)}$ is the corresponding value after applying plan $\mathcal{D}_k$. For mean travel time, the same construction is used:

$$\Delta \bar{\tau}_k^{(r)} = \bar{\tau}_k^{(r)} - \bar{\tau}_0^{(r)}. \quad (20)$$

The sign convention is kept simple: a positive paired difference means that the plan worsened the metric in that run and a negative value means that it improved it. The full vector of paired differences is kept, not only the average, because the consistency of an effect matters as much as its mean.

For a generic paired-difference vector $x_1, \dots, x_n$, the evaluator reports

$$\bar{x}, \quad s, \quad \min(x), \quad \max(x), \quad \tilde{x}, \quad (21)$$

where $\bar{x}$ is the sample mean, s the sample standard deviation and $\tilde{x}$ the median. The mean estimates the expected effect of the plan over the seed distribution, the standard

deviation describes run-to-run variability and the median is kept as a robust centre when a few runs are extreme.

The confidence interval answers a different question: how precisely the Monte Carlo experiment has estimated the mean paired effect. Because the true standard deviation of the paired effects is unknown and must be estimated from the n replications, the normal critical value is replaced by the Student- t critical value. The Student- t distribution has heavier tails than the normal distribution, so it gives wider intervals when only a small number of runs is available. The evaluator uses the standard 95% level, corresponding to a two-sided 5% error rate, as a conventional compromise between intervals that are too narrow to be reliable and intervals that are too wide to be useful. The interval is computed as

$$\bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}, \quad (22)$$

when at least two runs are available. In repeated experiments satisfying the usual independence assumptions, this procedure would contain the true mean effect in about 95% of such intervals, it does not mean that there is a 95% probability that this particular realized interval contains the truth.

Two diagnostic hypothesis tests are reported for each paired metric. Both ask whether the observed paired effects are compatible with a zero-effect baseline. The paired one-sample t -test checks the null hypothesis

$$H_0 : \mathbb{E}[\Delta M_k] = 0, \quad (23)$$

where ΔM_k is the paired effect of plan k on the metric being tested. It is most appropriate when the paired differences are approximately normally distributed. A Wilcoxon signed-rank test is also reported as a non-parametric companion. It ranks the absolute non-zero paired differences, attaches the original signs and checks whether positive and negative ranks are balanced around zero. This makes the check less dependent on the normality assumption, which is useful for traffic simulation outputs with small samples or occasional extreme runs.

3.5.4 Feasibility gate

Before plans are ranked, they pass through a completed-trip feasibility check. A candidate is considered feasible only if it completes at least as many trips as the baseline in every retained run:

$$C_k^{(r)} \geq C_0^{(r)} \quad \forall r \in \{1, \dots, n\}, \quad (24)$$

where $C_k^{(r)}$ is the number of completed trips under plan $\mathcal{D}_k$. The reason for this rule is straightforward: a detour should not look better simply because more vehicles failed to arrive before the end of the simulation window. Plans that fail the gate are still shown in the report, but they are not allowed into the Pareto screening or weighted score.

3.5.5 Impacted-vehicle metrics

Network-level metrics are necessary, but they can hide what happens to the drivers who are directly affected by the closure. For this reason, the evaluation also considers an impacted

vehicle set. When route files are available, a vehicle is counted as impacted for a plan if its edge sequence differs between the baseline route file and the plan-specific route file:

$$\mathcal{I}_k = \{v : R_{k,v} \neq R_{0,v}\}. \quad (25)$$

If route-difference information is not available, the evaluator can fall back to the vehicles whose baseline route crosses the closed edge. This is less precise, but it still keeps the analysis focused on users directly connected to the disruption.

For each retained run, the impacted-user analysis compares only vehicles that are present and completed in both the baseline and plan `tripinfo.xml` files. It reports the total additional travel time for impacted users,

$$\Delta T_{k,\mathcal{I}}^{(r)} = \sum_{v \in \mathcal{I}_k} (\tau_{k,v}^{(r)} - \tau_{0,v}^{(r)}), \quad (26)$$

the mean effective speed over their trajectories and the fraction of impacted users who are delayed:

$$\phi_k^{(r)} = \frac{|\{v \in \mathcal{I}_k : \tau_{k,v}^{(r)} > \tau_{0,v}^{(r)}\}|}{|\mathcal{I}_k|}. \quad (27)$$

The mean of $\phi_k^{(r)}$ across runs is used as one of the three decision criteria. Lower values are preferred, because they mean that a smaller share of directly affected users experiences a travel-time increase.

3.5.6 Multi-criteria decision analysis

The final ranking is built in three steps. First, infeasible plans are removed by the completed-trip gate of Section ???. Second, the remaining plans are screened for Pareto dominance using three cost criteria:

$$(\overline{\Delta D_k}, \overline{\Delta \tau_k}, \overline{\phi_k}). \quad (28)$$

A feasible plan a Pareto-dominates a feasible plan b if it is no worse on all three criteria and strictly better on at least one. Dominated plans are removed before the weights are applied. This prevents the final ranking from selecting a plan that is worse than another available plan on all measured dimensions.

Third, the plans left on the Pareto front are ranked with a normalized additive score. For each criterion q , values are min-max normalized over the Pareto front:

$$z_{kq} = \frac{x_{kq} - \min_j x_{jq}}{\max_j x_{jq} - \min_j x_{jq}}, \quad (29)$$

with $z_{kq} = 0$ when all surviving plans have the same value on criterion q . The composite score is then

$$S_k = \sum_q w_q z_{kq}, \quad (30)$$

where all criteria are costs and lower scores are better. The reference configuration gives equal weight to network time-loss delay, mean travel time and the fraction of impacted users delayed. This should be read as a neutral starting point, not as a claim that the three dimensions are inherently equally important.

Because the weights contain judgement, the evaluator also checks how sensitive the ranking is to them. It samples weight vectors uniformly from the three-criterion simplex using a Dirichlet(1, 1, 1) distribution. Here a weight vector is a triple (w_1, w_2, w_3) with non-negative entries that sum to one. For each sampled weight vector, the evaluator recomputes the ranking. The report then shows how often each Pareto-efficient plan ranks first, its mean rank, score quantiles and pairwise win probabilities. A plan that wins under the reference weights and also remains strong under many sampled weights is more convincing than a plan that only wins under a narrow weighting assumption.

4 Results

5 Discussion

5.1 Key findings

Present the key findings of the thesis.

5.2 Practical implications for urban planners

Present the practical implications for urban planners.

5.3 Limitations and validity checks

This section critically discusses limitations (including simulation realism) and frames these as threats to validity and explains how they might bias the evaluation or restrict generalization of findings.

5.4 Future work

Present the most impactful next steps for the work.

6 Conclusion